UNIT – IV

Transport-Level Security & Wireless Network Security

PART A: Transport Security

- Web Security Considerations
- SSL / TLS
- HTTPS
- Secure Shell (SSH)

PART B: Wireless Security

- Wireless Security Overview
- Mobile Device Security
- IEEE 802.11 Wireless LAN
- IEEE 802.11i Security

PART A: TRANSPORT-LEVEL SECURITY

1. Web Security Considerations

1.1 Introduction

The World Wide Web (WWW) is a global information system and the most visible part of the Internet. Web security refers to protecting web-based services, applications, and communications from threats. As web applications handle financial transactions, personal data, and business operations, security is paramount.

1.2 Web Security Threats

Threat	Description	Countermeasure
Integrity	Modification of user data, Trojan horse browser, modification of memory	Cryptographic checksums
Confidentiality	Eavesdropping on network traffic, theft of info from server, info about network config	Encryption, web proxies
Denial of Service	Killing user threads, flooding machine with bogus requests	Filtering, authentication
Authentication	Impersonation of legitimate users or servers, forged data	Cryptographic techniques
Non-repudiation	Denial of actions by user or server	Digital signatures

1.3 Web Communication Levels

Security can be provided at different layers of the protocol stack:

- **Network Layer (IPSec):** Transparent to applications; protects all traffic including HTTP
- **Transport Layer (SSL/TLS):** Most common; protects connection between browser and server; application-aware
- **Application Layer (S-HTTP):** Built into the HTTP protocol itself; less widely deployed

1.4 Key Web Security Requirements

- **Server Authentication:** Users need to verify they are communicating with the correct server
- **Client Authentication:** Servers may need to identify and verify the client
- **Data Integrity:** Data must not be modified in transit
- **Data Confidentiality:** Sensitive data must be encrypted in transit
- **Non-repudiation:** Actions cannot be denied after the fact

2. Secure Socket Layer (SSL) and Transport Layer Security (TLS)

2.1 Overview

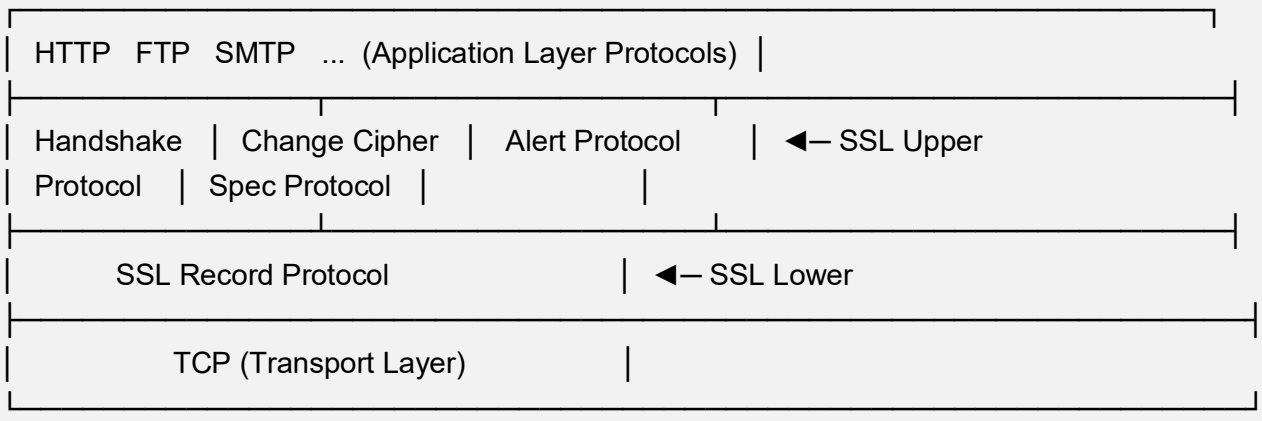
SSL was developed by Netscape in 1994 to provide security for web transactions. TLS (Transport Layer Security) is the successor to SSL, standardized by IETF. TLS 1.3 (RFC 8446, 2018) is the current standard and provides significant improvements over earlier versions.

Version	Status / Notes
SSL 2.0 (1995)	Deprecated — serious vulnerabilities, must not be used
SSL 3.0 (1996)	Deprecated — POODLE attack; must not be used
TLS 1.0 (1999)	Deprecated — based on SSL 3.0; BEAST attack
TLS 1.1 (2006)	Deprecated — minor improvements; still vulnerable
TLS 1.2 (2008)	Widely used; secure if configured correctly; SHA-256, AES-GCM
TLS 1.3 (2018)	Current standard; faster handshake; removes weak ciphers; 0-RTT

2.2 SSL/TLS Architecture

SSL/TLS operates between the Application Layer and the Transport Layer (TCP). It consists of two main layers:

SSL/TLS Protocol Stack



2.3 SSL Record Protocol

The SSL Record Protocol provides two services for SSL connections:

- **Confidentiality:** The Handshake Protocol defines a shared secret key used for conventional encryption of SSL payloads (AES, ChaCha20).
- **Message Integrity:** The Handshake Protocol defines a shared secret key used to form a MAC (HMAC-SHA256).

Record Protocol Operation

1. Fragment the application data into blocks of 2^14 bytes or less.
2. Optionally compress the data (usually disabled in practice).
3. Compute and append MAC: HMAC(MAC_write_key, seq_num || content_type || version || length || fragment).
4. Encrypt the data + MAC using the negotiated cipher (e.g., AES-128-GCM).
5. Prepend SSL Record header: content type, version, length.

2.4 SSL/TLS Handshake Protocol

The Handshake Protocol is the most complex part of SSL/TLS. It allows the server and client to authenticate each other and negotiate encryption algorithm and keys before data is sent.

TLS 1.2 Full Handshake (4 Phases)

Phase 1 – Establish Security Capabilities

- Client → Server: ClientHello (TLS version, random nonce, session ID, cipher suites, compression methods)
- Server → Client: ServerHello (chosen version, random nonce, session ID, chosen cipher suite)

Phase 2 – Server Authentication and Key Exchange

- Server → Client: Certificate (server's X.509 certificate chain)
- Server → Client: ServerKeyExchange (optional; for DHE/ECDHE key exchange)
- Server → Client: CertificateRequest (optional; for client authentication)
- Server → Client: ServerHelloDone

Phase 3 – Client Authentication and Key Exchange

- Client → Server: Certificate (optional; if requested by server)
- Client → Server: ClientKeyExchange (pre-master secret encrypted with server's public key, or DH params)
- Client → Server: CertificateVerify (optional; proves client has private key)

Phase 4 – Finish

- Client → Server: ChangeCipherSpec (signals switch to negotiated encryption)

- Client → Server: Finished (MAC of all handshake messages)
- Server → Client: ChangeCipherSpec
- Server → Client: Finished (verifies handshake integrity)

TLS 1.3 Improvements

- **1-RTT handshake:** Faster connection setup — only 1 round trip instead of 2 for full handshake
- **0-RTT resumption:** Session resumption can send data with zero round trips (with replay attack considerations)
- **Removed weak algorithms:** No RSA key exchange, no RC4, no DES, no SHA-1 in signatures, no CBC-mode ciphers
- **Forward Secrecy mandatory:** Only (EC)DHE key exchange; compromising long-term key cannot decrypt past sessions
- **Encrypted extensions:** More of the handshake is encrypted, reducing metadata leakage

2.5 Key Generation in TLS 1.2

After the key exchange, both sides compute:

6. Pre-master secret (PMS): from RSA or Diffie-Hellman exchange
7. Master secret: PRF(PMS, "master secret", ClientHello.random || ServerHello.random) → 48 bytes
8. Key material: PRF(master_secret, "key expansion", server_random || client_random) → key block
9. Key block split into: client_write_MAC_key, server_write_MAC_key, client_write_key, server_write_key, client_write_IV, server_write_IV

2.6 TLS Alert Protocol

The Alert Protocol conveys SSL-related alerts to the peer entity. Each message consists of two bytes:

- **Level (1 byte):** Warning (1) or Fatal (2). Fatal alerts cause immediate connection termination.
- **Alert code (1 byte):** Identifies the specific alert (e.g., close_notify, bad_certificate, handshake_failure, record_overflow).

3. HTTPS – HTTP over TLS

3.1 Overview

HTTPS (Hypertext Transfer Protocol Secure) is the combination of HTTP with SSL/TLS to provide encrypted communication and secure identification of web servers. It is defined in RFC 2818 and operates on port 443 by default.

3.2 How HTTPS Works

HTTPS Connection Establishment

10. User types https://www.example.com in browser
11. Browser connects to server on TCP port 443
12. TLS handshake is performed (server presents X.509 certificate)
13. Browser verifies certificate against trusted CAs in its certificate store
14. Session keys are generated and the secure tunnel is established
15. HTTP protocol operates normally inside the encrypted tunnel
16. All HTTP request/response data is encrypted and integrity-protected

3.3 HTTP vs HTTPS

Feature	HTTP	HTTPS
Port	80	443
Encryption	None — plaintext	TLS encryption
Data Integrity	Not guaranteed	MAC-protected
Authentication	None	Server certificate (mandatory), client cert (optional)
Performance	Faster (no crypto overhead)	Slightly slower (modern hardware minimizes this)
SEO / Browser	"Not Secure" warning in Chrome	Padlock icon; SEO ranking benefit
Used for	Internal/legacy systems	All modern websites, APIs, services

3.4 HTTPS Security Features

- **Server Authentication:** X.509 digital certificate proves server identity; prevents impersonation attacks
- **Encryption:** All data between browser and server is encrypted (AES-128-GCM or AES-256-GCM in TLS 1.3)
- **Integrity:** HMAC ensures data is not tampered with in transit; any modification is detectable
- **HSTS:** HTTP Strict Transport Security header forces browser to always use HTTPS for a domain
- **Certificate Pinning:** Mobile apps can pin specific certificates to prevent MITM even with rogue CAs

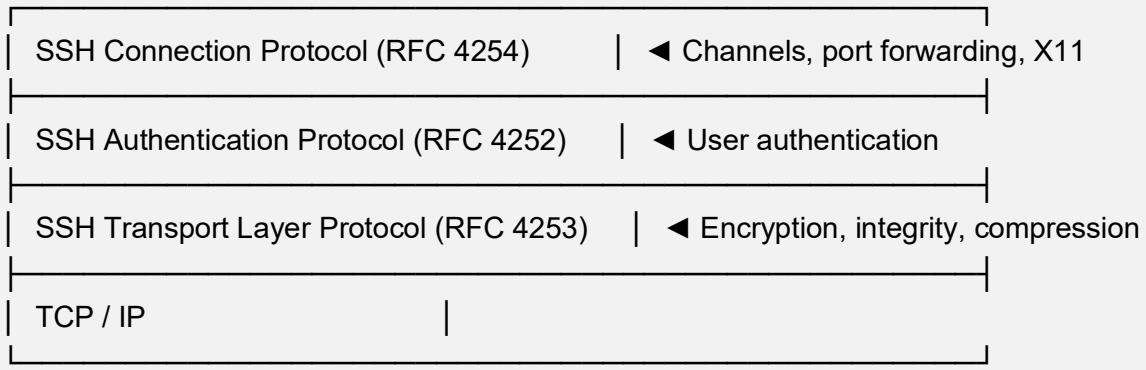
4. Secure Shell (SSH)

4.1 Introduction

Secure Shell (SSH) is a cryptographic network protocol for secure remote login, command execution, and file transfer over an insecure network. SSH was developed by Tatu Ylonen in 1995 to replace insecure protocols: Telnet, rlogin, rsh, and rcp. SSH-2 (RFC 4251–4254) is the current version.

4.2 SSH Architecture – Protocol Stack

SSH Protocol Layers



4.3 SSH Transport Layer Protocol

Provides server authentication, data confidentiality, and data integrity. Operates over TCP (default port 22).

Connection Setup Sequence

TCP connection established between client and server.

Protocol version exchange: both sides send 'SSH-2.0-<software>' string.

Algorithm negotiation (SSH_MSG_KEXINIT): key exchange algorithm, host key algorithm, encryption, MAC, compression.

Key exchange (e.g., Diffie-Hellman): generates shared secret K and exchange hash H.

Server sends its host key and signature; client verifies.

Session keys derived from K, H, and session ID.

New keys take effect; both sides send SSH_MSG_NEWKEYS.

4.4 SSH User Authentication Protocol

Authenticates the client-side user to the server. Supported methods:

Authentication Method	Description
Password	User sends username and password encrypted over SSH transport
Public Key	Client proves possession of private key; server verifies with stored public key (most secure)

Keyboard-Interactive	Server sends prompts; client responds (e.g., OTP, challenge-response)
GSSAPI / Kerberos	Delegated authentication using Kerberos tickets
Host-based	Trust based on client hostname + client user (deprecated; insecure)

Public Key Authentication Process

Client sends: username, service name, public key algorithm, public key.

Server checks if this public key is in ~/.ssh/authorized_keys.

Client creates and signs: session ID || message type || username || service || algorithm || public key.

Server verifies signature using the stored public key.

If valid, authentication succeeds.

4.6 SSH Connection Protocol

Provides multiple logical channels over a single SSH connection. Each channel is identified by a number and has flow control.

- **Interactive session:** Terminal access, command execution
- **X11 forwarding:** Secure tunneling of X Window System connections
- **TCP port forwarding (tunneling):** Local and remote port forwarding; can tunnel any TCP application
- **SFTP subsystem:** Secure file transfer over the same connection

PART B: WIRELESS NETWORK SECURITY

5. Wireless Security

5.1 Introduction

Wireless networks transmit data through radio waves, making them inherently more vulnerable to eavesdropping and attack than wired networks. Any device within radio range can potentially intercept wireless transmissions, making security mechanisms critical.

5.2 Wireless Network Types

Type	Standard	Range
WPAN (Personal)	Bluetooth, Zigbee, NFC	< 10 m
WLAN (Local)	IEEE 802.11 (Wi-Fi)	~100 m indoors
WMAN (Metropolitan)	WiMAX (802.16)	~50 km
WWAN (Wide)	4G LTE, 5G NR	Citywide / national

5.3 Wireless Threats

- **Eavesdropping (Passive Attack):** Attacker captures wireless traffic without active participation. No detection possible.
- **Traffic Analysis:** Even if encrypted, patterns of communication reveal useful information.
- **Jamming (DoS):** Attacker emits radio signals on the same frequency to disrupt communications.
- **Rogue Access Points:** Attacker sets up a fake AP to intercept client connections (Evil Twin attack).
- **Man-in-the-Middle:** Attacker intercepts and possibly modifies communications between two parties.
- **Replay Attacks:** Captured packets are retransmitted to gain access or disrupt services.
- **MAC Address Spoofing:** Attacker changes MAC address to bypass MAC-based access control.

5.4 Wireless Security Measures

- **Encryption:** WPA2 / WPA3 (Wi-Fi), TKIP, CCMP, AES to protect data in transit
- **Authentication:** 802.1X / EAP for enterprise authentication; WPA3-SAE for personal networks
- **SSID Management:** Disabling SSID broadcast (minor security); using non-default SSID names
- **MAC Filtering:** Allows only specific MAC addresses (weak — easily bypassed by spoofing)
- **Intrusion Detection:** Wireless IDS/IPS to detect rogue APs and suspicious traffic patterns
- **Segmentation:** Separate guest networks from corporate networks using VLANs

6. Mobile Device Security

6.1 Overview

Mobile devices (smartphones, tablets, wearables) are ubiquitous and carry sensitive personal and corporate data. They face unique security challenges due to their portability, constant connectivity, and diverse app ecosystems.

6.2 Mobile Security Threats

Threat	Description
Malicious Apps	Apps containing malware, spyware, or ransomware downloaded from app stores or sideloaded
Insecure Wi-Fi	Connecting to untrusted Wi-Fi networks exposes traffic to eavesdropping
Physical Theft/Loss	Lost/stolen devices expose stored data if not properly protected
OS Vulnerabilities	Unpatched vulnerabilities in Android/iOS exploited by attackers
Phishing	SMS phishing (smishing), email phishing targeting mobile users
Bluetooth Attacks	Bluejacking (spam), Bluesnarfing (data theft), Bluebugging (remote control)
IMSI Catchers (Stingrays)	Fake cell towers that intercept mobile communications
Rooting/Jailbreaking	Removing OS restrictions; exposes device to elevated risks

6.3 Mobile Security Controls

Device-Level Controls

- **Screen Lock:** PIN, password, pattern, biometric (fingerprint, face recognition)
- **Full Device Encryption:** Encrypts all data at rest; enabled by default on modern iOS and Android
- **Remote Wipe:** Erase device data remotely if lost or stolen (MDM, Find My iPhone, Android Device Manager)
- **Secure Boot:** Ensures only signed OS components load at startup

Application-Level Controls

- **App Sandboxing:** Each app runs in isolated environment; cannot access other apps' data
- **Permission Model:** Users grant specific permissions to apps (camera, contacts, location)
- **Code Signing:** Apps must be signed by registered developers; reduces malicious app distribution

- **App Store Review:** Apple App Store and Google Play scan apps for malware before publishing

Network-Level Controls

- **VPN:** Encrypts all device traffic when on untrusted networks
- **Certificate Pinning:** Apps verify specific server certificates; prevents MITM even with compromised CAs
- **TLS enforcement:** iOS App Transport Security (ATS), Android Network Security Config

6.4 Mobile Device Management (MDM)

MDM solutions (e.g., Microsoft Intune, VMware Workspace ONE) allow organizations to manage and secure employee mobile devices:

- Enforce security policies (minimum PIN length, encryption requirement)
- Distribute and manage apps remotely
- Wipe corporate data from personal devices (BYOD scenarios)
- Monitor compliance and detect rooted/jailbroken devices
- Enforce VPN use for corporate resource access

7. IEEE 802.11 Wireless LAN

7.1 Overview

IEEE 802.11 is the family of standards for wireless local area networks (WLANs), commonly known as Wi-Fi. Developed by the IEEE LAN/MAN Standards Committee, these standards define protocols for implementing wireless networking in the 2.4 GHz, 5 GHz, and 6 GHz frequency bands.

7.2 IEEE 802.11 Standard Evolution

Standard	Max Speed	Frequency / Notes
802.11 (1997)	2 Mbps	2.4 GHz; original standard
802.11b (1999)	11 Mbps	2.4 GHz; popularized Wi-Fi
802.11a (1999)	54 Mbps	5 GHz; less interference
802.11g (2003)	54 Mbps	2.4 GHz; backward compat with 802.11b
802.11n (2009) – Wi-Fi 4	600 Mbps	2.4/5 GHz; MIMO; 40 MHz channels
802.11ac (2013) – Wi-Fi 5	3.5 Gbps	5 GHz; MU-MIMO; 80/160 MHz
802.11ax (2019) – Wi-Fi 6/6E	9.6 Gbps	2.4/5/6 GHz; OFDMA; improved efficiency
802.11be (2024) – Wi-Fi 7	46 Gbps	2.4/5/6 GHz; MLO; 320 MHz channels

7.3 IEEE 802.11 Architecture

Network Components

- **Station (STA):** Any device with a wireless network interface card (laptop, phone, IoT device)
- **Access Point (AP):** Bridge between wireless and wired network; contains a Distribution System (DS) interface
- **Basic Service Set (BSS):** A group of stations that communicate with each other via a single AP (Infrastructure mode)
- **Independent BSS (IBSS):** Ad-hoc mode; stations communicate directly without an AP
- **Extended Service Set (ESS):** Two or more BSSes connected via a Distribution System; enables roaming
- **Distribution System (DS):** Backbone network (usually Ethernet) connecting multiple APs

7.4 IEEE 802.11 MAC Layer

Medium Access Control

802.11 uses CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) rather than CSMA/CD used in Ethernet, because collision detection is impractical in wireless.

- **DCF (Distributed Coordination Function):** Basic access method; uses CSMA/CA with random backoff
- **PCF (Point Coordination Function):** Optional polling-based method for time-bounded services; uses contention-free period
- **HCF (Hybrid Coordination Function):** Introduced in 802.11e for QoS; combines DCF and PCF

7.5 IEEE 802.11 Frame Format

Field	Description
Frame Control (2B)	Type (management/control/data), subtype, To DS, From DS, flags
Duration/ID (2B)	NAV value or association ID
Address 1 (6B)	Receiver address (RA)
Address 2 (6B)	Transmitter address (TA)
Address 3 (6B)	SA, DA, or BSSID depending on context
Sequence Control (2B)	Sequence number and fragment number for reassembly
Address 4 (6B)	Optional; used only in WDS (wireless bridges)
Frame Body (0–2304B)	Payload — varies by frame type
FCS (4B)	CRC-32 for error detection

7.6 Original 802.11 Security: WEP

Wired Equivalent Privacy (WEP) was the original security protocol for 802.11, designed to provide security equivalent to a wired LAN.

WEP Weaknesses (Why WEP is Broken)

- **Short IV (24 bits):** Initialization Vectors repeat after ~5000 packets; allows key recovery
- **Weak RC4 usage:** IV is prepended to key; certain IV values create weak keystreams (FMS attack)
- **No key management:** Static pre-shared key; all users share the same key
- **No replay protection:** Replayed packets are accepted
- **Weak integrity:** CRC-32 is not cryptographically secure; bit-flipping attacks possible

8. IEEE 802.11i Wireless LAN Security

8.1 Introduction

IEEE 802.11i (ratified in 2004) is the amendment to 802.11 that defines security mechanisms to replace the broken WEP. Wi-Fi Alliance branded 802.11i as WPA2 (Wi-Fi Protected Access 2). It defines the Robust Security Network (RSN) framework.

8.2 802.11i / RSN Overview

Security Services Provided by 802.11i

- **Authentication:** Via IEEE 802.1X/EAP (enterprise) or Pre-Shared Key (personal)
- **Key Management:** 4-Way Handshake and Group Key Handshake for key derivation and distribution
- **Data Confidentiality:** TKIP (transitional) or CCMP/AES (mandatory in WPA2)
- **Data Integrity:** Michael MIC (TKIP) or CCMP MIC (AES-based)
- **Replay Protection:** Sequence numbers (TSC) in CCMP prevent replay attacks

8.3 RSN Phases of Operation

Phase	Activities
1. Discovery	Client scans for APs; AP advertises security capabilities in Beacon/Probe frames (RSN IE)
2. Authentication	802.1X/EAP authentication or PSK validation with the AS (Authentication Server)
3. Key Management	4-Way Handshake derives PTK; Group Key Handshake derives GTK
4. Protected Data Transfer	Data exchanged using TKIP or CCMP encryption
5. Connection Termination	Disassociation or deauthentication; keys discarded

8.4 IEEE 802.1X Authentication

802.1X provides port-based Network Access Control. It involves three parties:

- **Supplicant:** The client device requesting access (STA)
- **Authenticator:** The network device controlling access (AP/switch)
- **Authentication Server:** RADIUS server that validates credentials (AS)

EAP over 802.1X Authentication Flow

17. Client associates with AP (no data access yet; controlled port blocked).
18. AP sends EAP-Request/Identity to client.
19. Client responds with EAP-Response/Identity (username).
20. AP forwards to RADIUS server (encapsulated in RADIUS Access-Request).
21. RADIUS challenges client via AP using selected EAP method.
22. Client and RADIUS server perform EAP authentication (EAP-TLS, EAP-TTLS, PEAP, etc.).
23. RADIUS sends Access-Accept with Master Session Key (MSK) to AP.
24. AP opens controlled port; 4-Way Handshake begins.

8.5 EAP Methods

EAP Method	Description
EAP-TLS	Mutual authentication with X.509 certificates (client + server); most secure; requires PKI
EAP-TTLS	Server cert only; tunnels inner auth (PAP, CHAP, MSCHAPv2) inside TLS tunnel
PEAP (Protected EAP)	Server cert only; tunnels MSCHAPv2 or GTC; widely used in enterprises
EAP-FAST	Cisco's alternative to PEAP; uses Protected Access Credentials (PAC) instead of certificates
EAP-SIM / EAP-AKA	Authentication using SIM card credentials; used in cellular/Wi-Fi integration

8.6 Key Hierarchy in 802.11i

Pairwise Key Hierarchy (Unicast Traffic)

Key Derivation Chain

- Pre-Shared Key (PSK) or Master Session Key (MSK) from 802.1X → Pairwise Master Key (PMK)
- PMK + ANonce + SNonce + MACs → via PRF-512 → Pairwise Transient Key (PTK)

- PTK split into:
  - KCK (Key Confirmation Key) – 128 bits: EAPOL-Key MIC verification
  - KEK (Key Encryption Key) – 128 bits: Key data encryption in 4-way handshake
  - TK (Temporal Key) – 128 bits: Actual data encryption (AES-CCMP)

Group Key Hierarchy (Broadcast/Multicast Traffic)

- **GMK (Group Master Key):** Generated by AP
- **GTK (Group Temporal Key):** Derived from GMK; distributed to all clients via Group Key Handshake; encrypted with KEK

8.7 The 4-Way Handshake

The 4-Way Handshake establishes the PTK between the AP and a client. It confirms that both parties possess the PMK, derives fresh session keys, and installs them.

4-Way Handshake Messages

25. **Message 1 (AP→STA):** AP sends ANonce (AP's random nonce). STA can now compute PTK.
26. **Message 2 (STA→AP):** STA sends SNonce + MIC (computed with KCK). AP computes PTK and verifies MIC.
27. **Message 3 (AP→STA):** AP sends GTK (encrypted with KEK) + MIC. STA verifies and installs PTK and GTK.
28. **Message 4 (STA→AP):** STA confirms key installation. AP installs PTK.

> **Note:** KRACK (Key Reinstallation Attack, 2017) exploited the 4-Way Handshake by forcing nonce reuse via replay of handshake messages. Patched in all modern systems.

8.8 CCMP – Counter Mode with CBC-MAC Protocol

CCMP is the mandatory encryption protocol in WPA2/802.11i. It is based on AES operating in CCM (Counter Mode + CBC-MAC) mode and provides both confidentiality and integrity.

CCMP Frame Structure

Component	Details
Block cipher	AES with 128-bit key
Confidentiality	AES Counter (CTR) mode — keystream XORed with plaintext
Integrity / Auth	AES CBC-MAC — 8-byte MIC over header + plaintext
IV / Nonce	48-bit Packet Number (PN) — prevents replay attacks
Header overhead	8 bytes CCMP header + 8 bytes MIC = 16 bytes per MPDU

AAD	Certain unencrypted fields (addresses) included in MIC computation

CCMP Encryption Process

29. Construct CCMP header with Packet Number (PN) and key ID.
30. Compute MIC over AAD (unencrypted header fields) + MPDU data using CBC-MAC mode.
31. Encrypt MPDU data + MIC using CTR mode with nonce derived from PN.
32. Transmit: MAC Header | CCMP Header | Encrypted Data | Encrypted MIC.

8.9 TKIP (Transitional Security Protocol)

TKIP (Temporal Key Integrity Protocol) was introduced as an interim fix for WEP, designed to run on legacy WEP hardware. It was defined in WPA (Wi-Fi Protected Access) before 802.11i was finalized.

8.10 WPA3 Enhancements

- **SAE (Simultaneous Authentication of Equals):** Replaces PSK; based on Dragonfly key exchange (ECC); resistant to offline dictionary attacks and PMKID attacks
- **Forward Secrecy:** SAE provides session-unique keys; compromising one session doesn't expose others
- **Protected Management Frames (PMF):** Mandatory in WPA3; prevents deauthentication attacks and forgery of management frames
- **WPA3-Enterprise 192-bit:** Uses 192-bit minimum security suite (GCMP-256, BIP-GMAC-256, ECDHE) for sensitive environments
- **Easy Connect:** QR code-based device provisioning for IoT devices (Wi-Fi Device Provisioning Protocol — DPP)

QUICK REVISION SUMMARY

Topic Summary Table

Topic	Key Concept	Key Detail
Web Security	Multi-layer threat model	Integrity, Confidentiality, DoS, Authentication
SSL Record Protocol	Encryption + MAC over TCP	Fragment → MAC → Encrypt → Header
TLS Handshake	Key exchange + authentication	4 phases; certificate-based server auth
TLS 1.3	Faster, stronger than 1.2	1-RTT, mandatory FS, removed weak ciphers
HTTPS	HTTP over TLS (port 443)	Server cert verification + AES-GCM encryption
SSH Transport	Encrypted remote access	DH key exchange; replaces Telnet/rlogin
SSH Auth	Multiple auth methods	Public key auth most secure
Wireless Threats	Eavesdropping, MITM, rogue APs	No physical boundary = attack surface
Mobile Security	Multi-layer protection	Sandboxing, encryption, MDM, VPN
802.11 MAC	CSMA/CA medium access	RTS/CTS, ACK, backoff mechanism
WEP	Broken — never use	24-bit IV reuse; RC4 keystream recovery
802.11i / WPA2	RSN with AES-CCMP	4-Way Handshake; PTK/GTK key hierarchy
CCMP	AES CCM mode	Counter mode encryption + CBC-MAC integrity
TKIP	Deprecated RC4 fix	Per-packet keys; Michael MIC; still weak
WPA3	SAE + Forward Secrecy	Dragonfly; protected mgmt frames; PMF mandatory

Important Port Numbers

Protocol	Port	Description
HTTP	80/TCP	Unencrypted web traffic

HTTPS	443/TCP	HTTP over TLS
SSH	22/TCP	Secure Shell
RADIUS	1812/UDP (auth), 1813/UDP (acct)	Authentication server for 802.1X
SFTP	22/TCP	SSH file transfer (over SSH)

Key Acronyms

Acronym	Full Form	Context
TLS	Transport Layer Security	SSL successor; current web security standard
HTTPS	HTTP Secure	Web browsing with TLS
SSH	Secure Shell	Remote login / file transfer
WPA2	Wi-Fi Protected Access 2	802.11i implementation
CCMP	Counter Mode with CBC-MAC Protocol	AES-based 802.11i encryption
TKIP	Temporal Key Integrity Protocol	Deprecated WEP replacement
PTK	Pairwise Transient Key	Unicast session key in 802.11i
GTK	Group Temporal Key	Broadcast/multicast key in 802.11i
SAE	Simultaneous Authentication of Equals	WPA3 personal authentication
PMF	Protected Management Frames	WPA3 mandatory; prevents deauth attacks
EAP	Extensible Authentication Protocol	Framework used in 802.1X
RSN	Robust Security Network	802.11i security framework

Exam Tips

- SSL operates between TCP and Application layers; TLS is its successor.
- TLS 1.3 uses only ECDHE/DHE key exchange — RSA key exchange is removed.
- SSH has 3 protocols: Transport Layer, User Authentication, Connection.
- WEP is broken due to 24-bit IV and static key — never used in modern systems.
- 802.11i = WPA2; uses CCMP (AES) as mandatory encryption.
- 4-Way Handshake: derives PTK from PMK + ANonce + SNonce + MAC addresses.
- WPA3 uses SAE (Dragonfly) which provides forward secrecy — WPA2-PSK does not.
- CCMP uses 48-bit Packet Number (PN) to prevent replay attacks.